

CHROME EXTENSION · MANIFEST V3

Find in Images

Making scanned PDFs and image-based documents searchable with the browser's native Ctrl+F, through on-device OCR and an invisible DOM text overlay.



Technical Documentation & Design Rationale

JavaScript ES6+

PDF.js

Tesseract.js

IndexedDB

Canvas / DOM

Version 1.0 · Architecture, implementation & phase-by-phase build narrative

Contents

1 Introduction

2 Problem Statement

3 Goals, Scope & Constraints

3.1 Goals

3.2 The read-only constraint

3.3 Non-goals

4 Functionalities & User Stories

5 System Architecture

5.1 Pipeline overview

5.2 Component map

1 Introduction

Find in Images is a Chrome (Manifest V3) extension that makes *any* PDF behave like a real, searchable web page — including scanned documents and PDFs whose text lives inside pictures. When you open a PDF and press `Ctrl+F`, the browser's own find feature locates words that are physically part of an image, not just selectable text.

The core idea is deliberately conservative: rather than building a bespoke search engine, the extension **transforms the document into something the browser already knows how to search**. It renders each page, recognises the text inside images with on-device OCR, and injects that text back into the page as invisible, precisely-positioned DOM elements. The browser's native find then "just works", complete with its familiar highlighting and match navigation.

Design philosophy. Lean on native browser capabilities instead of reinventing them. We never implement search indexing, query parsing, or highlight logic — we make the content searchable and let Chrome do the rest. Everything runs locally; no document ever leaves the machine.

This document explains the problem, the architecture, every technology used and exactly how it is integrated, a walkthrough of the implementation, and — importantly — the **phase-by-phase reasoning** behind how the system was built, including what each phase set out to solve and why it led naturally to the next.

2 Problem Statement

PDFs and document images fall into two broad categories:

- **Text-based** — the characters are encoded as text. The browser can already select and search them.
- **Image-based** — scanned pages, photographed documents, screenshots, or "text" that has been flattened into a picture. To a computer these are just coloured pixels: **not selectable, not searchable**.

Chrome's built-in PDF viewer renders image-based PDFs perfectly but cannot find a single word inside them, because there is no text to find. For anyone working with scanned reports, lecture notes, invoices, or archival material, this is a daily friction point: the information is visible but not reachable.

A further subtlety: many real PDFs are **mixed**. A single page may contain native text *and* images that themselves contain text (a scanned figure with a caption, a table rendered as a picture, a logo with a tagline). A useful solution must handle native text, image text, and the two together on the same page — without duplicating words or misaligning them.

The problem in one line. Make the text that lives *inside images* findable through the browser's native `Ctrl+F`, on any PDF, without altering the original file.

3 Goals, Scope & Constraints

3.1 Goals

- **Accurate, findable image text.** The headline feature: text inside images must be reliably detectable via `Ctrl+F`. This is the project's top priority.
- **Faithful rendering.** The PDF must look like a PDF — crisp on high-DPI screens, fit to the window, not like a low-resolution scan.
- **Native search experience.** Searching, highlighting, and match-to-match navigation should feel built-in.
- **Responsiveness at scale.** Large PDFs must remain usable — opening, scrolling, and searching should never block on full-document processing.
- **Offline and private.** No network calls; OCR and language models run entirely on-device.
- **Selectable and copyable text.** Users must be able to select and copy text, and what they copy must match what they see.

3.2 The read-only constraint

Hard constraint — the original PDF is never modified. The solution is strictly **read-only render + overlay**. The extension renders a faithful view of the file and lays an invisible text layer *on top*; it never rewrites, re-encodes, or changes the format of the source document in any way. Every architectural decision respects this boundary.

This constraint is the reason we build a custom viewer at all (Section 5) and the reason OCR output is stored separately in a cache (Section 8.6) rather than written back into the document.

3.3 Non-goals

- Editing, annotating, or re-saving PDFs.
- Server-side or cloud OCR (everything is on-device by design).
- Perfect OCR. OCR is inherently imperfect; the system is designed to *maximise findability despite* recognition errors (see correction variants, Section 8.7) rather than to guarantee character-perfect transcription.
- Searching arbitrary images on ordinary web pages (the content script is a stub reserved for a future phase).

4 Functionalities & User Stories

The behaviour is best understood as a set of user stories, each mapped to the capability that satisfies it.

User story	How the system satisfies it
As a user , when I open a PDF link, I want it to open in a viewer that can search image text — automatically, with no extra clicks.	A Manifest V3 service worker installs a <code>declarativeNetRequest</code> rule that transparently redirects every <code>.pdf</code> navigation to the bundled viewer (8.1).
As a user , I want the document to look sharp and fit my window, not zoomed or pixelated.	PDF.js renders each page fit-to-width, with a device-pixel-ratio-scaled canvas backing store for crisp HiDPI output (8.2, 8.4).
As a user , I want to press <code>Ctrl+F</code> and find words that are part of an image.	Image regions are OCR'd with Tesseract.js and the recognised words are injected as an invisible, aligned text layer that native find can match (8.3, 8.7).
As a user , I want to jump between matches and see the active one clearly, even on colourful pages.	A custom find bar draws translucent highlight boxes with a distinct, pulsing active match, navigable by the up/down buttons, Enter/Shift+Enter, F3, or arrow keys (8.7, 11).
As a user , I want to select and copy text, and have the copied text match what I see.	Text spans are measured and horizontally scaled to the exact glyph width, so selection ranges and clipboard output line up with the rendering (9).
As a user with a long PDF, I don't want to wait for the whole file before I can scroll or search.	Pages render lazily as they approach the viewport; the rest are indexed in the background during idle time across parallel OCR workers (12).
As a user , I don't want to re-pay the OCR cost every time I reopen the same document.	Recognised words are cached in IndexedDB, scoped per document <i>and</i> per OCR language, with a format version for safe invalidation (8.6).
As a non-English user , I want OCR in my language.	A language selector (English, Danish, or both) drives Tesseract with bundled, offline trained data; results are cached independently per language (8.3).
As a user with a scanned document that has faint or blurry text, I still want it found.	Before OCR, each region is rendered at ~360 DPI and passed through a contrast stretch + unsharp mask to recover low-contrast and blurred glyphs (8.5).

5 System Architecture

The extension is a **pipeline** that converts an opaque PDF into searchable DOM text. Conceptually, content flows from a PDF request, through interception and rendering, into a text-detection fork (native text vs. OCR), and finally onto an invisible overlay that the browser searches natively.

5.1 Pipeline overview

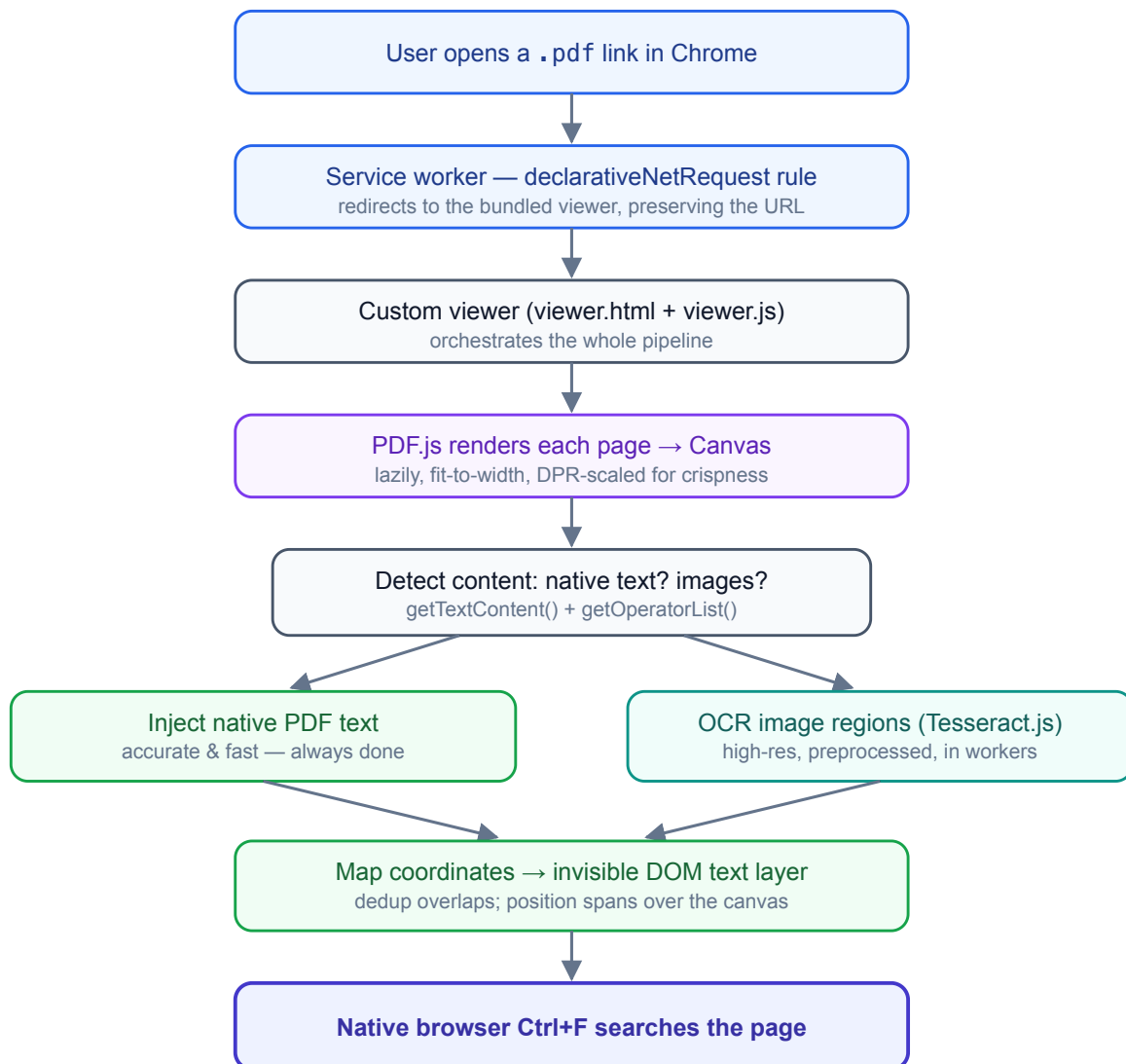


Figure 1 — End-to-end pipeline. The fork in the middle is the heart of the system: native text and OCR'd image text are produced independently, then merged onto one overlay.

5.2 Component map

The codebase is organised in four layers: the extension runtime, the custom viewer application, the vendored libraries, and the browser platform APIs they build on.

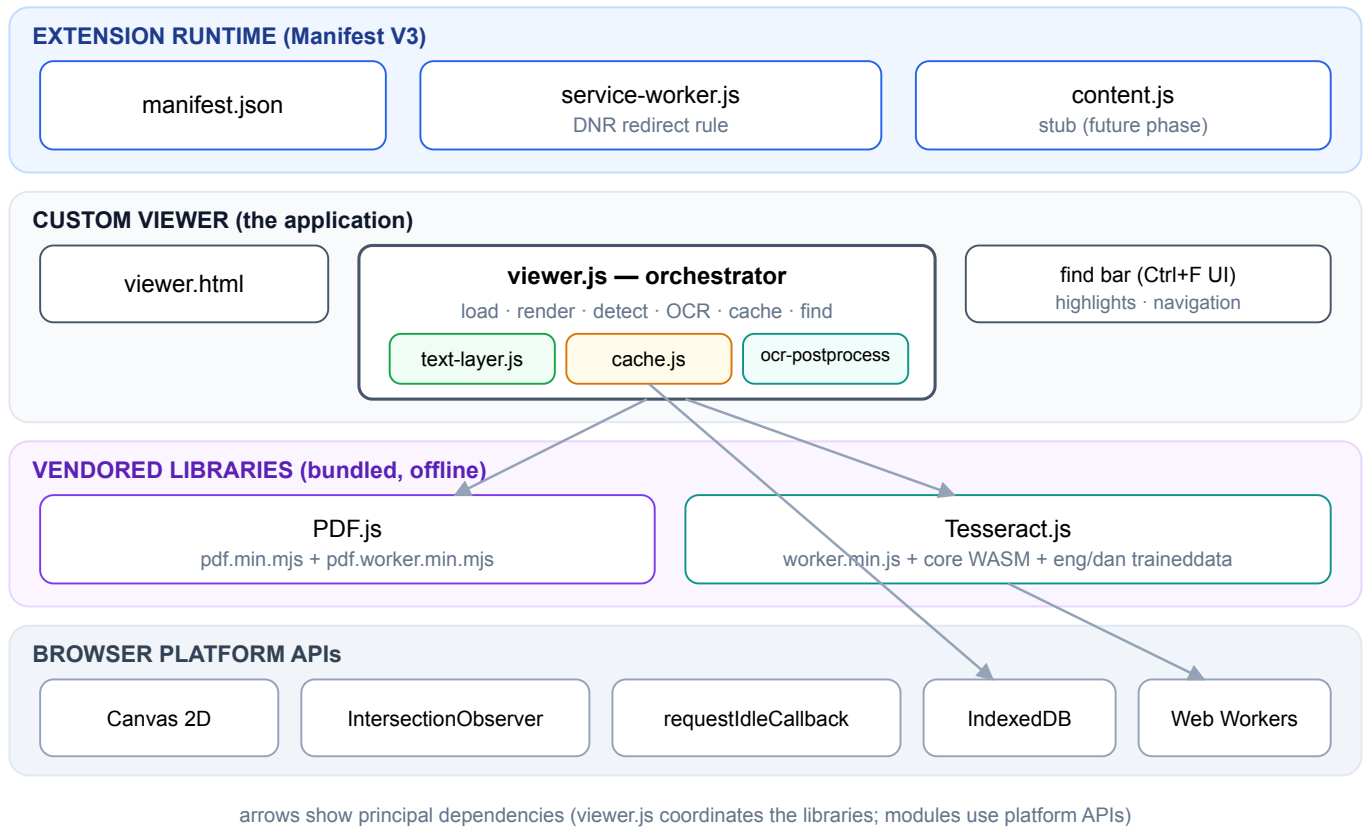


Figure 2 — Component map. `viewer.js` is the orchestrator; thin modules handle text-layer geometry, caching, and OCR clean-up; the heavy lifting is delegated to two vendored libraries that run entirely offline.